



Universidad Experimental De Guayana

Vicerrectorado Académico

Coordinación General De Pregrado

Proyecto De Carrera: Ingeniería En Informática

Asignatura: Lenguajes y Compiladores

Los Lenguajes de Programación

Profesor:

Márquez Félix

Alumnos:

C.I.:

Aguilera Rhixeidys V- 30.851.503

Curbata Emilio V- 28.214.578

Bravo Yvanna V- 28.530.717

Albornoz Ronniel V- 24.889.173

Ciudad Guayana, junio de 2026

Resumen Académico

El presente estudio analiza la infraestructura lingüística y computacional de los lenguajes de programación, evaluando cómo las decisiones de diseño morfológico, léxico y sintáctico impactan la eficiencia del software. Se examinaron cuatro lenguajes representativos de distintos paradigmas y mecanismos de ejecución: Zig, Rust, Python y JavaScript. Para contrastar su rendimiento, se implementó un algoritmo de procesamiento intensivo basado en el cálculo de la ecuación de segundo grado sobre vectores de gran volumen ($n = 200$). Los resultados del *benchmarking* empírico demostraron variaciones significativas en los tiempos de ejecución y consumo de memoria, evidenciando las diferencias arquitectónicas entre lenguajes de compilación nativa (Rust, Zig), entornos interpretados (Python) y motores de compilación *Just-In-Time* (JavaScript). Finalmente, como propuesta de innovación arquitectónica, se diseñó un Lenguaje de Dominio Específico (DSL) denominado "Lenguaje L", orientado a la interfaz hombre-máquina de ECO-GRID. La rigurosa definición de su alfabeto, reglas léxicas y gramática abstracta permitió codificar programas eficientes para la prevención de fugas térmicas y el balance autónomo de carga, consolidando la comprensión de los lenguajes como herramientas de software moldeables a necesidades industriales críticas.

Introducción

En el ámbito de la ciencia de la computación, un lenguaje de programación trasciende la noción de ser un simple medio para escribir algoritmos; constituye en sí mismo un producto de software complejo con una arquitectura minuciosamente diseñada. Históricamente, el estudio de la programación suele enfocarse en la perspectiva del usuario final, priorizando la traducción de lógica para resolver problemas estructurados. Sin embargo, para dominar el diseño de compiladores, es fundamental abordar el planteamiento conceptual de la infraestructura lingüística. Esta infraestructura se sostiene sobre niveles formales: el nivel morfológico y léxico, encargados de la estructura de los símbolos y su agrupación en *tokens* con significado semántico; y el nivel sintáctico, que establece la jerarquía y el orden gramatical para formar sentencias lógicas sin ambigüedad. En la actualidad, analizar la eficiencia estructural de estos niveles es de vital importancia. Comprender cómo las fases de un intérprete o compilador procesan el código fuente permite a los ingenieros no solo seleccionar la herramienta adecuada para optimizar el rendimiento, sino también diseñar lenguajes de dominio específico (DSL) capaces de interactuar con precisión y seguridad en entornos ciberfísicos críticos.

1. Diferentes Paradigmas de los Lenguajes de Programación

En la ingeniería de software moderna, los lenguajes de programación han dejado de ser herramientas puristas adscritas a una sola filosofía. Esta evolución surge de la necesidad de construir sistemas escalables y mantenibles, demostrando cómo lenguajes de amplio propósito general adoptan características funcionales u orientadas a objetos simultáneamente para responder a las demandas del mercado.

Por ejemplo, al construir interfaces de usuario complejas (como las que se desarrollan en arquitecturas frontend modernas), resulta sumamente útil combinar la organización estructural de la Programación Orientada a Objetos con la inmutabilidad y la transparencia del paradigma funcional. Esto permite que lenguajes tradicionalmente orientados a objetos (como C# o Java) o dinámicos (como JavaScript y Python) incorporen funciones de orden superior, cierres léxicos (*closures*) y evaluación perezosa para manejar la asincronía y el estado global de forma mucho más segura. La convergencia no es una pérdida de identidad del lenguaje, sino una optimización arquitectónica para resolver problemas específicos con la herramienta conceptual más eficiente.

1.1 Matriz Descriptiva de Paradigmas Clásicos y Emergentes

Paradigma	Ejes Temáticos	Análisis Crítico y Comportamiento
Imperativo / Estructural	Gestión explícita del estado del sistema, secuenciación de instrucciones, mutabilidad de memoria y efectos secundarios.	Es el modelo más cercano a la arquitectura de hardware tradicional (Von Neumann). Al permitir la mutabilidad directa, ofrece un alto rendimiento, pero delega en el programador la responsabilidad total de controlar los efectos colaterales, lo que en sistemas grandes complica la depuración y el mantenimiento.
Orientado a Objetos (POO)	Modelos de encapsulamiento, polimorfismo, herencia VS. composición, y abstracción	Modela el software mapeando entidades del mundo real. Actualmente, el debate crítico se

	basada en datos y comportamiento combinados.	inclina hacia priorizar la "composición sobre la herencia" para evitar jerarquías rígidas y frágiles, fomentando un acoplamiento más débil y sistemas más modulares.
Funcional	Inmutabilidad de datos, funciones como ciudadanos de primer orden, evaluación perezosa (lazy evaluation), transparencia referencial y eliminación programática de efectos colaterales.	Al prohibir el cambio de estado (inmutabilidad) y garantizar que una función siempre retorne el mismo valor para los mismos argumentos (transparencia referencial), elimina variables ocultas. Esto hace que el código sea inherentemente predecible y altamente paralelizable sin riesgo de colisiones.
Lógico / Declarativo	Programación basada en relaciones, unificación, resolución de cláusulas de Horn y abstracción total del flujo de control por parte del programador.	El programador describe el "qué" (el problema y sus reglas) en lugar del "cómo" (los pasos para resolverlo). El motor de inferencia asume el control del flujo, lo que resulta excepcionalmente potente para sistemas expertos, inteligencia artificial deductiva y validación de reglas complejas.
Concurrente / Actores (Emergente)	Modelos de paso de mensajes, aislamiento estricto de estado entre entidades concurrentes y mitigación de condiciones de carrera a nivel de diseño lingüístico.	Aborda la principal limitación de la programación moderna: la sincronización de hilos. Al garantizar un aislamiento estricto y comunicarse únicamente mediante paso de mensajes, elimina la necesidad de bloqueos (<i>locks</i>) y previene las condiciones de carrera desde la raíz del lenguaje.

2. Estudio Morfológico, Sintáctico y Benchmarking

2.1 Zig

- **Análisis Morfológico (Léxico)**

Zig es un lenguaje de **forma libre**: espacios y saltos de línea solo separan tokens.

Los bloques se delimitan con `{}` y las sentencias terminan en `;`.

Categorías de tokens:

Categoría	Ejemplos
Palabras reservadas (~48)	const, var, fn, pub, if, else, while, try, return
Identificadores	[A-Za-z_][A-Za-z0-9_]*; forma extendida @"..." para nombres arbitrarios
Builtins	@sqrt, @import, @as, @floatFromInt — funciones intrínsecas del compilador
Literales enteros	200, 100_000 (el _ es separador visual)
Literales flotantes	1.0, 4.0, 0.0
Cadenas	"texto\n"
undefined	Memoria no inicializada (var a: [N]f64 = undefined)
Operadores	+, -, *, /, ==, >, +=, .*
Puntuadores	(,), [,], {, }, :, ;, ,

Comentarios

- // — comentario de línea (único tipo ordinario).
- /// — documentación de declaración. //! — documentación de módulo.
- **No existen** /* ... */: decisión de diseño que simplifica el lexer.

Zig intencionalmente mantiene un conjunto pequeño de palabras clave. Comportamientos que otros lenguajes hacen implícitos (defer, errdefer, comptime, try) son tokens explícitos, haciendo el flujo de control predecible.

- **Análisis Sintáctico**

Zig usa una gramática libre de contexto imperativa, similar a C. Las construcciones principales son:

- **Declaraciones de variables**

```
VarDecl ::= ( "const" | "var" ) Identificador [ ":" Tipo ] "=" Expr  
";"
```

- const → enlace **immutable** (el compilador rechaza reasignación).
- var → **mutable**. Zig obliga a usar const cuando no se reasigna.

```
const N: usize = 200;  
var a: [N]f64 = undefined;
```

- **Definición de funciones**

```
FnDecl ::= [ "pub" ] "fn" Nombre "(" [ Param { "," Param } ] ")"  
ReturnType Bloque
```

El ! en el tipo de retorno (!void) indica **error union**: el llamador debe manejar el error con try o catch.

```
pub fn main() !void { ... }  
fn solveAll(a: *const [N]f64, ...) f64 { ... }
```

- **Condicional (if / else)**

```
IfExpr ::= "if" "(" Expr ")" Bloque { "else" "if" "(" Expr ")"  
Bloque } [ "else" Bloque ]
```

```
if (disc > 0.0) { ... }  
else if (disc == 0.0) { ... }  
else { ... }
```

- **Repetición (while)**

```
WhileExpr ::= "while" "(" Expr ")" [ ":" "(" Expr ")" ] Bloque
```

El : (i += 1) es la **continue expression**: se ejecuta al final de cada iteración (equivale al tercer campo del for de C).

```
while (i < N) : (i += 1) { ... }
```

El for de Zig se reserva para iterar sobre slices/arreglos con captura de elemento (for (slice) |elem| { ... }), no para bucles con contador.

2.2 Python

- **Análisis Morfológico (Léxico) de la Solución:**

El analizador léxico de Python (lexer) descompone el flujo de caracteres de este programa en unidades mínimas con significado semántico (*tokens*), aplicando las siguientes reglas de producción formal:

- **Identificadores:** Los nombres de variables y subprogramas (*calcular_ecuacion_segundo_grado*, *discriminante*, *soluciones*) cumplen con la

expresión regular clásica del lenguaje: $[a - zA - Z_][a - zA - Z0 - 9_]*$. Se adoptó el estándar morfológico *snake_case*.

- **Palabras Reservadas:** Se aíslan componentes gramaticales invariables y protegidos del compilador/intérprete, tales como *import, def, for, in, if, else, y return*.
- **Literales:** El lexer clasifica constantes numéricas de tipo entero (200, 4, 2) y de punto flotante (1.0, 10.0, 50.0, 0.5).
- **Operadores y Signos de Puntuación:** Reconocimiento de operadores aritméticos de asignación (=), exponenciación (^), multiplicación (*), resta (-), suma (+) y división (/), junto a operadores relacionales de comparación (>=).
- **Análisis del Espacio en Blanco (Regla del Off-side):** Python no utiliza delimitadores léxicos como ; o llaves {}. El lexer mide los espacios al inicio de cada línea física para inyectar de forma sintética tokens virtuales *INDENT* (incremento de sangría) y *DEDENT* (decremento de sangría). Esto fuerza a que la morfología defina directamente la estructura de bloques del programa.
- **Análisis Sintáctico y Árbol de Sintaxis Abstracta (AST):**

Una vez generada la corriente de tokens, el *parser* valida el cumplimiento jerárquico de la gramática. El script implementado somete el flujo a una estructura sintáctica estrictamente anidada:

1. **Bloque Funcional Superior:** Declaración del bloque *def* que parametriza la lógica algorítmica.
2. **Estructura de Control Iterativa:** Invocación de un bucle determinado *for* subordinado a la secuencia generada por el iterable *range(n)*.

3. **Estructura de Control Condicional:** Bifurcación sintáctica *if – else* que evalúa la expresión lógica del discriminante.

A nivel de compilación interna, esta secuencia produce un Árbol de Sintaxis Abstracta (AST) donde la operación matemática de la raíz cuadrada y el cálculo cuadrático se evalúan como subárboles de expresiones aritméticas compuestas, asegurando la precedencia de operadores antes de la asignación final en la lista.

2.3 Rust

- **Análisis Morfológico (Léxico)**

Rust es un lenguaje de forma libre: espacios y saltos de línea solo separan tokens. Los bloques se delimitan con { } y las sentencias terminan en ;.

- **Categorías de tokens**

Categoría	Ejemplos
Palabras reservadas (~39)	let, fn, pub, if, else, while, for, return, use, const, mut, struct, impl, match
Identificadores	[a-zA-Z_][a-zA-Z0-9_]*; convención snake_case para variables y funciones
Literales enteros	200, 100_000 (el _ es separador visual), 0u32, 0u64 (con sufijo de tipo)
Literales flotantes	0.0f64, 1000.0, 4.0 (con sufijo de tipo opcional)
Cadenas	"texto\n"
Operadores	+, -, *, /, ==, >, +, .. (rango exclusivo), ..= (rango inclusivo)
Puntuadores	(,), [,], {, }, :, ;, ,, :: (separador de módulo)
Macros	println!, format! — se distinguen por el ! al final del identificador

Comentarios

- // — comentario de línea ordinario.
- /// — documentación de declaración (genera docs con cargo doc).

- `//!` — documentación de módulo.
- `/* ... */` — comentario de bloque (a diferencia de Zig, Rust sí los soporta).

- **Observaciones léxicas destacadas**

Rust distingue entre variables (`let`) inmutables por defecto y mutables (`let mut`), convirtiendo la mutabilidad en un token explícito que el compilador verifica estáticamente. Las macros se diferencian léxicamente de las funciones por el símbolo `!`, lo que simplifica el análisis sintáctico. El compilador rechaza en tiempo de compilación cualquier reasignación a un enlace inmutable, haciendo el flujo de datos predecible desde la fase léxica.

- **Análisis Sintáctico**

Rust usa una gramática libre de contexto con influencias funcionales e imperativas. A continuación, se documentan las construcciones principales empleadas en el código del benchmark.

- **Declaraciones de variables**

Regla de producción:

```
LetDecl ::= "let" [ "mut" ] Identificador [ ":" Tipo ] "=" Expr ";"
```

`let` → enlace inmutable por defecto (el compilador rechaza reasignación). `let mut` → mutable. Rust obliga a declarar explícitamente la mutabilidad, siendo este un principio de diseño central del lenguaje.

Ejemplos del código:

```
let mut state = SEED;
let mut a = [0.0f64; N];
let elapsed = start.elapsed();
```

- **Definición de funciones**

Regla de producción:

```
FnDecl ::= [ "pub" ] "fn" Nombre "(" [ Param { "," Param } ] ")" [ "->" Tipo ] Bloque
```

El `->` indica el tipo de retorno. Si se omite, la función retorna `()` (unit type). Las referencias se anotan con `&` (inmutable) y `&mut` (mutable), siendo parte fundamental del sistema de ownership de Rust.

Ejemplos del código:

```
fn lcg(state: &mut u64) -> f64 { ... }
fn solve_all(a: &[f64; N], b: &[f64; N], c: &[f64; N]) -> f64 { ... }
fn main() { ... }
```

- **Condicional (if / else)**

Regla de producción:

```
IfExpr ::= "if" Expr Bloque { "else" "if" Expr Bloque } [ "else" Bloque ]
```

A diferencia de C y Zig, Rust no requiere paréntesis alrededor de la condición. El `if` en Rust es una expresión, no una sentencia, por lo que puede retornar un valor.

Ejemplos del código:

```
if disc > 0.0 { ... }
else if disc == 0.0 { ... }
else { ... }
```

- **Repetición (for / while)**

Reglas de producción:

```
ForExpr    ::= "for" Patrón "in" Expr Bloque
WhileExpr  ::= "while" Expr Bloque
```

Rust usa `for` con iteradores para recorrer rangos. El rango `0..N` es exclusivo (no incluye `N`); `0..=N` es inclusivo. El patrón `_` descarta la variable de iteración cuando no se necesita.

Ejemplos del código:

```
for i in 0..N { ... }           // itera i = 0, 1, ..., N-1
for _ in 0..REPS { ... }       // descarta la variable de iteración
loop { a[i] = lcg(...); if a[i].abs() > 0.001 { break; } }
```

- **Diferencia clave con Zig**

En Rust el `for` se usa directamente para bucles con contador mediante rangos (`0..N`), mientras que en Zig el `for` se reserva exclusivamente para iterar slices y el contador se implementa con `while`. Rust también dispone de `loop` (bucle infinito con `break` explícito), que Zig no tiene como palabra reservada separada.

2.4 JavaScript

- **Análisis Morfológico (Léxico) de JavaScript:**

El analizador léxico (lexer) de JavaScript procesa el código fuente transformándolo en una secuencia de tokens válidos. Basado en el algoritmo de la ecuación cuadrática desarrollado, el comportamiento léxico se define de la siguiente manera:

- **Palabras Reservadas (Keywords):** El lenguaje restringe estrictamente tokens como `const`, `let`, `function`, `if`, `else`, `return` y `require` para definir la estructura de control y gestión de memoria.
- **Reglas de Identificadores:** Las variables y funciones (ej. `generarVector`, `resolverEcuacion`, `vectorA`) siguen la convención *camelCase*. El motor exige que comiencen con una letra, guion bajo (`_`) o el símbolo de dólar (`$`), siendo completamente sensibles a mayúsculas y minúsculas (case-sensitive).

- **Tipos de Literales:** El escáner identifica literales numéricos enteros (200, 4, 2), de punto flotante generados por Math.random(), literales de cadena de texto ("Raíz Compleja") y literales de estructuras de datos como los arreglos vacíos ([]).
- **Tratamiento de Elementos Irrelevantes y Delimitadores:** A diferencia de lenguajes como Python, JavaScript considera los espacios en blanco, tabulaciones y saltos de línea (la indentación) como elementos léxicos irrelevantes. El lenguaje utiliza delimitadores explícitos: llaves {} para definir ámbitos de bloque (block scope) y paréntesis () para evaluar expresiones. Aunque cuenta con Inserción Automática de Punto y Coma (ASI), las sentencias en este algoritmo se cierran explícitamente con; para evitar ambigüedades en la tokenización.
- **Análisis Sintáctico y Jerarquía de Estructuras:**

A nivel sintáctico, el código construye un Árbol de Sintaxis Abstracta (AST) jerárquico basado en las estructuras de control esenciales.

La jerarquía del algoritmo se fundamenta en las siguientes estructuras:

1. **Definición de Subprogramas:** El nivel más alto del contexto de ejecución lo dictan las declaraciones de función (function Nombre(parametros) {...}), que encapsulan la lógica algorítmica.
2. **Estructuras Repetitivas (Bucles):** Se emplea el bucle for, cuya jerarquía sintáctica exige tres expresiones separadas por punto y coma dentro de paréntesis, seguidas de un bloque de ejecución.
3. **Estructuras Condicionales:** Se utiliza una cascada if ... else if ... else para la bifurcación lógica dependiendo del valor del discriminante.

2.5 Tabla de Tiempos

Lenguaje de Programación	Paradigma Dominante	Mecanismo de Ejecución y Compilación	Tiempo de Ejecución Promedio (ms)	Consumo de Memoria Pico (MB)
Zig	Imperativo / Estructurado	Compilación Nativa (LLVM)	~66 ms	~1.23 MB
Python	Multiparadigma (OO, Imperativo)	Interpretado (CPython / VM)	2.7749 ms	0.023041 MB
Rust	Multiparadigma (Funcional, Imperativo)	Compilación Nativa (LLVM)	0.000723212 ms	4800 MB
JavaScript	Multiparadigma (Prototípico, Funcional)	JIT (Just-In-Time) / V8 Engine	2.2811 ms	0.0681 MB

2.6 Algoritmo

El algoritmo seleccionado fue: Cálculo de ecuación de segundo grado para vectores a,b,c con cantidad de elemento n para n=200.

- Justificación de la elección
 - **Estrés de CPU mediante Punto Flotante:** La fórmula general $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ requiere operaciones matemáticas complejas por debajo del capó, como el cálculo de raíces cuadradas y potencias. Al iterar esto 200 veces, se genera un volumen de

procesamiento matemático suficiente para contrastar empíricamente la velocidad bruta de los lenguajes compilados nativamente (como Rust y Zig) frente a los que dependen de máquinas virtuales o compilación JIT (como Python y JavaScript).

- **Contraste en la Gestión de Memoria:** El algoritmo exige la creación, almacenamiento y manipulación de múltiples arreglos (vectores a, b, c, más el vector de resultados) en la memoria RAM. Esto expone una de las diferencias arquitectónicas más importantes que están estudiando: cómo impacta el rendimiento el uso de un Recolector de Basura (en Python y JavaScript) en comparación con la gestión de "propiedad" (*ownership*) de Rust o el control manual de Zig.
- **Equilibrio entre Lógica y Análisis Sintáctico:** A diferencia de simular una *blockchain* (que tiene una mayor complejidad estructural), la ecuación cuadrática posee una lógica matemática universal y transparente. Esto garantiza que el equipo no pierda tiempo depurando la lógica del negocio y pueda enfocarse exclusivamente en lo que evalúa la asignación: el análisis morfológico, la sintaxis y el rendimiento de cada entorno.
- **Determinismo y Verificabilidad:** Al procesar los mismos vectores base, el resultado matemático debe ser exactamente el mismo en los cuatro lenguajes. Esto facilita la verificación de que todos los miembros del equipo implementaron el código correctamente antes de medir los tiempos.

En resumen, es un algoritmo que castiga tanto al procesador como a la memoria lo suficiente como para sacar a relucir las verdaderas diferencias entre los paradigmas, manteniendo un código limpio y legible.

3. Diseño de un Lenguaje de Dominio Específico (DSL) para Sistemas logico – físicos

3.1 Programa de Ejemplo — Escenario Operativo A: Escenario Operativo A (Prevención de Fuga Térmica y Gestión de Alivio de Carga)

```
# =====
# PROGRAMA A - ECO-GRID - Prevencion de Fuga Termica y
Gestion de Alivio de Carga
# Lenguaje L |
# Escenario Operativo A: Monitoreo termico critico y proteccion
del banco de baterias
# =====

init_grid

    # --- Umbrales y parametros de seguridad termica ---
    UMBRAL_CRITICO_TEMP = 55          # grados Celsius:
temperatura de peligro termico
    UMBRAL_TEMP_NORMAL = 40          # grados Celsius:
temperatura de retorno a operacion normal
    MAX_CICLOS_PELIGRO = 5          # ciclos maximos tolerados
antes de emitir emergencia total
    banco = banco_1          # banco de baterias
principal bajo monitoreo
    ciclos_en_peligro = 0          # contador de ciclos
consecutivos con temperatura critica

    activo = VERDADERO

    # --- Lazo de monitoreo termico continuo ---
    mientras activo == VERDADERO ejecutar

        temp = leer_temperatura(banco);          # lectura del
sensor termico de celda del banco

        # CASO CRITICO: temperatura supera el umbral de
seguridad -> riesgo de fuga termica
        si_verdadero temp > UMBRAL_CRITICO_TEMP entonces

            ciclos_en_peligro = ciclos_en_peligro + 1;

            # Acciones inmediatas de proteccion (orden
determinista)
            activar_refrigeracion(banco);
# 1. encender sistema de refrigeracion auxiliar
```

```

        conmutar_linea(linea_solar_carga, AISLADO);
# 2. desconectar ingreso de carga solar
        conmutar_linea(sector_industrial, AISLADO);
# 3. aislar sector industrial de la microred
        conmutar_linea(sector_industrial_respaldo,
CONECTADO); # 4. desviar consumo a red comercial de respaldo

        emitir_alerta("ALERTA TERMICA: temperatura critica
detectada en banco de baterias");

        # Verificar si el peligro persiste tras el periodo
maximo tolerado
        si_verdadero ciclos_en_peligro >= MAX_CICLOS_PELIGRO
entonces
            emitir_alerta("EMERGENCIA CRITICA: fuga termica
persistente - intervencion manual requerida");
            activo = FALSO; # detener el ciclo
automatico para forzar intervencion humana
            fin_si

        sino

            # Temperatura dentro del rango operativo seguro
            si_verdadero ciclos_en_peligro > 0 entonces

                # La temperatura bajo: restituir estado normal
de la planta
                conmutar_linea(linea_solar_carga, CONECTADO);
# reconectar carga solar
                conmutar_linea(sector_industrial, CONECTADO);
# restituir sector industrial a microred
                conmutar_linea(sector_industrial_respaldo,
AISLADO); # desconectar respaldo comercial
                ciclos_en_peligro = 0;
                emitir_alerta("Temperatura normalizada: planta
restituida a operacion normal");

            sino
                emitir_alerta("Monitoreo activo: temperatura en
rango seguro");
            fin_si

        fin_si

        esperar(60); # re-evaluar el estado
termico cada 60 segundos

    fin_mientras

terminar

```

3.2 Programa de Ejemplo — Escenario Operativo B: (Balance de Carga y Optimización Energética Autónoma)

```
# =====
# PROGRAMA B - ECO-GRID - Balance de Carga y Optimizacion
Energetica Autonoma
# Lenguaje L | Ronniel e Yvanna
# Escenario Operativo B: Gestion autonoma de excedentes y
proteccion nocturna
# =====

init_grid

# --- Umbrales de decision ---
UMBRAL_OPTIMO      = 90          # % carga para considerar
excedente vendible
UMBRAL_CRITICO     = 20          # % carga minimo de seguridad
nocturna
UMBRAL_TEMP_SEGURA = 45          # grados Celsius: maximo seguro
para inyeccion a red
banco              = banco_1     # banco de baterias principal

activo = VERDADERO

# --- Lazo de optimizacion energetica autonoma (evaluacion
continua) ---
mientras activo == VERDADERO ejecutar

carga      = estado_carga(banco);          # % de carga actual de
las baterias
generacion = flujo_generacion();           # kW generados por los
paneles solares
demanda    = demanda_actual();             # kW demandados por la
planta
hora       = hora_actual();                # hora del sistema
(0..23)

# CASO 1: carga optima Y generacion supera la demanda -> vender
excedente a la red
si_verdadero carga > UMBRAL_OPTIMO Y generacion > demanda
entonces

temp = leer_temperatura(banco);            # verificar temperatura
antes de inyectar

si_verdadero temp < UMBRAL_TEMP_SEGURA entonces
excedente = generacion - demanda;
conmutar_linea(rele_venta, CONECTADO);
```

```

inyectar_red(excedente);
emitir_alerta("Excedente energetico: inyectando kW a la red
publica");
sino
emitir_alerta("Temperatura elevada en bateria: inyeccion
suspendida por seguridad");
fin_si;

sino

# CASO 2: carga critica EN horario nocturno de alta demanda
si_verdadero carga < UMBRAL_CRITICO Y (hora >= 18 O hora <= 6)
entonces

# Preservar suministro critico: aislar sectores no esenciales
conmutar_linea(sector_no_esencial, AISLADO);
conmutar_linea(sector_medico, CONECTADO);
conmutar_linea(sector_servidores, CONECTADO);
emitir_alerta("Carga critica nocturna: aislando sectores no
esenciales");

sino

# Estado intermedio: operacion estable, sin accion de balance
emitir_alerta("Operacion estable: sin accion de balance
requerida");

fin_si

fin_si

esperar(60);                # re-evaluar cada 60 segundos

fin_mientras

terminar

```

- **Estructura Sintáctica del Lenguaje L**

- Notación EBNF simplificada de las construcciones usadas en el Escenario B:

```

Programa      ::= "init_grid" Sentencia* "terminar"

Sentencia     ::= Condicional
                | Bucle
                | Accion ";"
                | Asignacion ";"

```

```

Asignacion      ::= IDENT "=" Expr

Condicional     ::= "si_verdadero" Cond "entonces" Sentencia*
                  [ "sino" Sentencia* ] "fin_si"

Bucle           ::= "mientras" Cond "ejecutar" Sentencia*
"fin_mientras"

Cond            ::= Expr OPREL Expr
                  | Cond OPLOG Cond
                  | "NO" Cond
                  | "(" Cond ")"

Accion          ::= Llamada
Llamada         ::= IDENT "(" [ Args ] ")"
Args            ::= Expr { "," Expr }

Expr            ::= Termino { OPARIT Termino }
Termino         ::= IDENT | ENTERO | DECIMAL | BOOLEANO | ESTADO
| CADENA
                  | Llamada | "(" Expr ")"

```

– **Regla de no ambigüedad:**

Toda estructura de control abre con su palabra clave (`si_verdadero`, `mientras`) y cierra con su marca explícita (`fin_si`, `fin_mientras`).

Las acciones y asignaciones terminan en `;`. Los bloques no dependen de la indentación.

- **Explicación lógico-sintáctica**

1. `init_grid` ... `terminar` delimitan el programa (regla raíz de la gramática).
2. El bucle ``mientras ... ejecutar ... fin_mientras`` modela el carácter autónomo y continuo del balance: re-evalúa cada 60 s.
3. Cada iteración lee cuatro **sensores** (`estado_carga`, `flujo_generacion`, `demanda_actual`, `hora_actual`) y asigna sus valores a variables locales.
4. **Condicional principal** `carga > UMBRAL_OPTIMO Y generacion > demanda`: el operador lógico `Y` combina dos condiciones relacionales. Si se cumple,

se verifica la temperatura con `leer_temperatura` antes de inyectar — protección de seguridad adicional.

5. **Condional anidado** en la rama `sino: carga < UMBRAL_CRITICO Y (hora >= 18 O hora <= 6)`. Los paréntesis agrupan la sub-condición horaria, evitando ambigüedad de precedencia. Aquí se aíslan sectores no esenciales y se mantienen conectados el sector médico y de servidores.
6. La rama `sino` interna cubre el estado intermedio: ni venta ni emergencia, el sistema sigue monitoreando.

Conclusión

La investigación demuestra de manera concluyente que las decisiones teóricas en el diseño gramatical y léxico de un lenguaje de programación tienen un impacto directo y medible en su desempeño frente a cargas de procesamiento intensivo. Al analizar empíricamente las estructuras de Zig, Rust, Python y JavaScript, se evidenció cómo los mecanismos subyacentes de cada paradigma determinan drásticamente los tiempos de ejecución y la huella de memoria. El *benchmarking* del algoritmo cuadrático corroboró que lenguajes con restricciones formales estrictas y compilación nativa facilitan una gestión de recursos superior frente a arquitecturas puramente dinámicas o interpretadas. Asimismo, el desarrollo del "Lenguaje L" para la gestión de la microred ECO-GRID validó que la abstracción sintáctica constituye una necesidad industrial práctica. Diseñar este Lenguaje de Dominio Específico exigió aplicar rigurosamente las teorías de morfología léxica para delimitar un vocabulario coherente e inequívoco. Esto demuestra que la seguridad y eficiencia operativa de un sistema de control automatizado dependen intrínsecamente de un diseño lingüístico libre de ambigüedades. En síntesis, el dominio profundo de las fases de compilación e interpretación faculta al ingeniero informático para superar el rol de consumidor de sintaxis y asumir el diseño arquitectónico de las herramientas tecnológicas del futuro.

Referencias Bibliográficas

- Stroustrup, B. (2013). El lenguaje de programación C++ (4.a ed.). Addison-Wesley.
- El equipo de Rust. (sf). El lenguaje de programación Rust . Recuperado de

<https://doc.rust-lang.org>

- Hutton, G. (2016). Programación en Haskell (2.a ed.). Cambridge University Press.
- Vaca Rodríguez, C. (2011). Paradigmas de programación [Apuntes de cátedra].

Universidad de Valladolid.

- Aho, AV, Lam, MS, Sethi, R. y Ullman, JD (2007). Compiladores: Principios, técnicas y herramientas (2.a ed.). Pearson.